



SC6531 Camera and LCD Driver Adaptation Guide

Issue V1.0
Date 07-Nov-2023

Copyright © UNISOC (Shanghai) Technologies Co., Ltd. All rights reserved.

All data and information contained in or disclosed by this document is confidential and proprietary information of UNISOC (Shanghai) Technologies Co., Ltd. (hereafter referred as UNISOC) and all rights therein are expressly reserved. This document is provided for reference purpose, no license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document, and no express and implied warranties, including but without limitation, the implied warranties of fitness for any particular purpose, and non-infringement, as well as any performance. By accepting this material, the recipient agrees that the material and the information contained therein is to be held in confidence and in trust and will not be used, copied, reproduced in whole or in part, nor its contents revealed in any manner to others without the express written permission of UNISOC. UNISOC may make any changes at any time without prior notice. Although every reasonable effort is made to present current and accurate information, UNISOC makes no guarantees of any kind with respect to the matters addressed in this document. In no event shall UNISOC be responsible or liable, directly or indirectly, for any damage or loss caused or alleged to be caused by or in connection with the use of or reliance on any such content.

Please refer to the UNISOC Documents in the UNISOC Deliverables for the use of the Deliverables. Any loss caused by the modification, customization or use of the UNISOC Deliverables in violation of the instructions in the UNISOC Documents shall be undertaken by those who conduct so. The performance indicators, test results and parameters in the UNISOC Deliverables are all obtained in the internal development and test system of UNISOC and are only for the reference. Before using UNISOC Deliverables commercially or conducting mass production of the Deliverables, comprehensive testing and debugging in combination with its own software and hardware test environment are pre-requisite.

Trademarks

UNISOC, , Spreadtrum, SPRD, RDA, and other UNISOC trademarks are trademarks of UNISOC (Shanghai) Technologies Co., Ltd. and/or its affiliated companies.

All other trademarks and trade names mentioned in this document are the property of their respective owners.

Disclaimer

This document may include third-party content, including but not limited to third-party information, software, components, and data. UNISOC does not control or assume responsibility for any third-party content, including but not limited to its accuracy, compatibility, reliability, availability, legitimacy, appropriateness, performance, non-infringement, and status update, unless otherwise specified in this document. UNISOC does not provide any guarantee or authorization for the third-party content mentioned or referenced in this document.

User should check the availability of third-party content. If a third-party license is needed, user shall obtain it in an authorized or legal way, unless otherwise specified in this document.

UNISOC (Shanghai) Technologies Co., Ltd.



About this document

Purpose

This document mainly introduces how to adapt Camera Sensor and LCD drivers on the SC6531 platform. The content includes driver adaptation and debugging methods.

Intended audience

This document is intended primarily for the engineers needing to adapt Camera Sensor and LCD drivers on the SC6531 platform.

Acronyms and abbreviations

Acronym and abbreviation	Full name
ESD	Electro Static Discharge

Symbol conventions

The symbols that may be found in this guide are defined in the following table.

Symbol	Description
 NOTE	Calls attention to important information, best practices and tips. NOTE is used to address information not related to personal injury, equipment damage, and environment deterioration.
 CAUTION	Calls attention to error-prone operations. CAUTION is used to address information not related to personal injury, equipment damage, and environment deterioration.
 WARNING	Calls attention to irreversible operations. WARNING is used to address information not related to personal injury and environment deterioration.

Revision history

Issue	Date	Description
V1.0	07-Nov-2023	This issue is the first official release

Keywords

Camera Sensor, LCD, driver adaptation

Contents

1 Camera Sensor	1
1.1 Driver adaptation	1
1.2 Debugged Sensors on the platform	2
1.3 Debugging method	2
1.3.1 Sensor ID not identified	2
1.3.2 Fail to open the camera	3
2 LCD	4
2.1 Driver adaptation	4
2.2 Debugging method	6
2.2.1 LCD ID not identified	6
2.2.2 Abnormal LCD display	6

1 Camera Sensor

Take sensor_gc032A as an example to introduce how to adapt and debug the new Camera Sensor driver on the SC6531 platform.

1.1 Driver adaptation

To adapt a new Camera Sensor driver, the following configurations need to be completed:

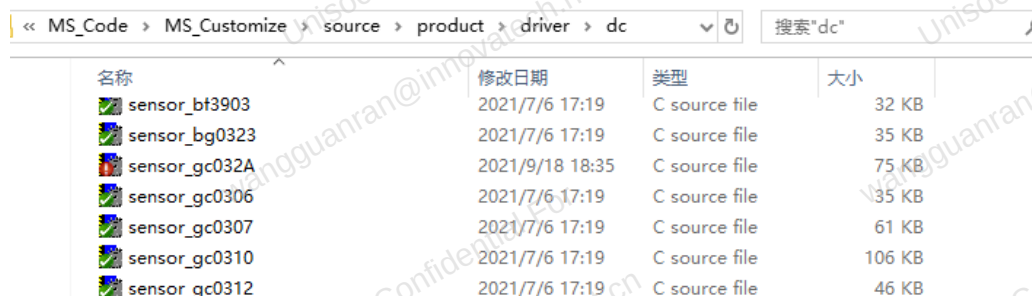
- Configure GPIO IOMUX in pinmap_cfg.c.

Path: MS_Code\MS_Customize\source\product\config\fp6531_bar\pinmap_cfg.c

- Add the Camera Sensor driver file.

- Save the Camera Sensor driver file to the following path:

MS_Code\MS_Customize\source\product\driver\dc



名称	修改日期	类型	大小
sensor_bf3903	2021/7/6 17:19	C source file	32 KB
sensor_bg0323	2021/7/6 17:19	C source file	35 KB
sensor_gc032A	2021/9/18 18:35	C source file	75 KB
sensor_gc0306	2021/7/6 17:19	C source file	35 KB
sensor_gc0307	2021/7/6 17:19	C source file	61 KB
sensor_gc0310	2021/7/6 17:19	C source file	106 KB
sensor_gc0312	2021/7/6 17:19	C source file	46 KB

- Add sensor_gc032A.c in custom_drv.mk.

Path: MS_Code\make\custom_drv\custom_drv.mk

```
SOURCES += sensor_ov7670.c \
sensor_bf3703.c \
sensor_bf3903.c \
sensor_bf30a2.c \
sensor_gc6113.c \
sensor_gc2015.c \
sensor_gc032A.c \
sensor_NT99250.c \
sensor_gc2145.c
```

- Add the information of the new Camera Sensor in sensor_cfg.c.

Path: MS_Code\MS_Customize\source\product\config\sensor_cfg.c

```
//extern const SENSOR_INFO_T g_SIV121A_yuv_info;
extern const SENSOR_INFO_T g_BF3903_yuv_info;
extern const SENSOR_INFO_T g_GC032A_yuv_info;
```

```
extern const SENSOR_INFO_T g_GC0310_yuv_info;
.....
LOCAL const SENSOR_INFO_T *main_sensor_infor_tab[] = {
    //&g_GC0306_yuv_info,
    &g_GC032A_yuv_info,
    //&g_SIV121A_yuv_info,
    &g_GC0310_yuv_info,
    PNULL
};
```

1.2 Debugged Sensors on the platform

SC6531 platform supports parallel interface (CCIR601) and serial interface (SPI). The new Camera Sensor needs to support the two interfaces. Currently, the debugged Camera Sensors on the platform are as follows:

- GC0310
- GC032A
- BF3903
- BF30a2
- BF20a6

1.3 Debugging method

1.3.1 Sensor ID not identified

After adding the Camera Sensor driver, if Sensor ID is identified, the following keywords can be found in the logs:

- DCAMERA_Open: the camera is opened

17:29:23.087	--	DCAMERA_Open mode 1
17:29:23.087	--	DCAMERA_SetSensorType: 1
17:29:23.087	--	SENSOR: Sensor_Init start 218107
17:29:23.087	--	SENSOR: Sensor_Init is done\n
17:29:23.087	--	SENSOR: Sensor_Init the index of sensor 0 is 2
17:29:23.087	--	SENSOR: Sensor_Init the index of sensor 1 is 255
17:29:23.087	--	SENSOR: Sensor_Init the index of sensor 2 is 255
17:29:23.087	--	Sensor_SetCurId : 0, start time: 218108

- Identify: Sensor ID is identified

```

17:29:23.087 -- SENSOR: Sensor_SetMode src = 0x0, divd = 0x0
17:29:23.087 -- set_BF20A6_Power_On
17:29:23.087 -- set_BF20A6_Power_On:1
17:29:23.087 -- SENSOR: Sensor_SetMCLK -> s_sensor_mclk = 12MHz, clk = 12MHz
17:29:23.087 -- SENSOR: Sensor_SetMCLK -> Do nothing !!
17:29:23.087 -- set_BF20A6_Power_On
17:29:23.087 -- BF20A6_Identify
17:29:23.087 -- SENSOR: IIC read reg OK! 0x00fc, val:0x0020
17:29:23.087 -- SENSOR: IIC read reg OK! 0x00fd, val:0x00a6
17:29:23.087 -- BF20A6_Identify: it is BF20A6
17:29:23.087 -- SENSOR: Sensor_SetMode -> mode = 0, time 218176
17:29:23.087 -- SENSOR: Sensor_SetMCLK -> s_sensor_mclk = 12MHz, clk = 24MHz
17:29:23.087 -- SENSOR: Sensor_SetMCLK src = 0x2, divd = 0x1
17:29:23.087 -- SENSOR: Sensor_SendRegValueToSensor -> reg_count = 140 start time = 218176
17:29:23.087 -- SENSOR: IIC write reg OK! 0x00f2, val:0x0001
17:29:23.087 -- SENSOR: IIC write reg OK! 0x0012, val:0x0010
17:29:23.087 -- SENSOR: IIC write reg OK! 0x0002, val:0x0000

```

If Sensor ID is not identified, debug the camera sensor driver in the following ways:

- Sensor is not powered on
Sensor cannot be powered on until the two sensor pins of AVDD and IOVDD are powered on at the same time. Use a multimeter to directly measure the voltage of these two pins to determine whether the sensor is powered on.
- I2C has a problem
Check whether the signal of SCL/SDA sensor pin meets timing sequence requirements through the oscilloscope.

1.3.2 Fail to open the camera

After identifying Sensor ID, if you open the camera and enter the preview successfully, the following keywords can be found in the logs:

DCAMERA_StartPreview: enter the preview

```

17:29:23.087 -- [MMIDC] out_angle = 0
17:29:23.087 -- [MMIDC] MMIAPIDC_GetDCModuleSensorAngle angle=3,id=0
17:29:23.087 -- [MMIDC] *angle = 3
17:29:23.087 -- [MMIDC] GetDCPreviewParam lcd_width = 240, lcd_height = 320
17:29:23.087 -- [MMIDCSRV] MMIDCSRV_GetDCDisplayMode disp_mode=1
17:29:23.087 -- [MMIDC] GetDCPreviewParam rotation = 3, display mode = 1
17:29:23.087 -- DC: [DCAMERA_StartPreview] -> start time = 218280
17:29:23.087 -- _DCAMERA_GetMutex
17:29:23.087 -- ISP_SERVICE: ISP_ServiceOpen, service = 0, state 255, cur_time 218280, wait_opt 4000
17:29:23.087 -- ISP_SERVICE: ISP_ServiceOpen, Got controller,cur_time 218281 thread id 21
17:29:23.087 -- ISP_SERVICE: queue info:enqueued:0, store: 50, count: 1
17:29:23.087 -- ISP_Service: _ISP_ServiceMainMessageLoop,command=0x1 param=0x0 SignalCode 0x0 sig_pt
17:29:23.087 -- _DCAMERA_PutMutex
17:29:23.087 -- DC: display region , x 0,y 0,w 240,h 320

```

If Sensor ID is identified successfully, but the camera fails to be opened, debug the camera sensor driver in the following ways:

- Measure whether the camera pin signal is normal and meets the timing sequence requirements.
- Confirm whether the driver file is correct and the sensor output data conforms to the interface protocol (SPI/DVP).

2 LCD

Take tft_GC9307 as an example to introduce how to adapt and debug the new LCD driver on the SC6531 platform.

2.1 Driver adaptation

- Add the LCD driver file.
 - Save the LCD driver file to the following path:
MS_Code\MS_Customize\source\product\driver\lcd

MS_Code > MS_Customize > source > product > driver > lcd					搜索"lcd"
名称	修改日期	类型	大小		
FSTN_NT7538.C	2021/10/27 13:57	C source file	16 KB		
lcd_backlight.c	2021/10/27 13:57	C source file	15 KB		
lcd_diag.c	2021/10/27 13:57	C source file	14 KB		
tft_GC9104.c	2021/10/27 13:57	C source file	33 KB		
tft_GC9106.c	2021/10/27 13:57	C source file	31 KB		
tft_GC9304C.c	2021/10/27 13:57	C source file	22 KB		
tft_GC9307.c	2021/10/27 13:57	C source file	33 KB		
tft_hx8312a.c	2021/10/27 13:57	C source file	21 KB		
tft_HX8340B.c	2021/10/27 13:57	C source file	17 KB		

- Add tft_GC9307.c in custom_drv.mk.
Path: **MS_Code\make\custom_drv\custom_drv.mk**

LCD DRIVER

```
SOURCES += tft_s6d0139.c \
          tft_hx8312a.c \
          tft_ILI9320.c \
          tft_ILI9328.c \
          tft_ILI9325.c \
          tft_ILI9326.c \
          ...
          tft_NV3029g.c \
          tft_HX8340B.c \
          tft_ST7789H2.c \
          tft_GC9307.c
```

- Modify the configuration file.
 - Add the structure declaration in **MS_Code\MS_Customize\export\inc\lcm_cfg.h**.

```
#define LCD_DRV_ID_ILI9225      0x9225
#define LCD_DRV_ID_ILI9320      0x9320
#define LCD_DRV_ID_ILI9325      0x9325
#define LCD_DRV_ID_ILI9326      0x9326
#define LCD_DRV_ID_GC9307      0x9307
.....

extern const LCD_SPEC_T g_lcd_HX8312[];
extern const LCD_SPEC_T g_lcd_HX8346[];
extern const LCD_SPEC_T g_lcd_HX8347[];
extern const LCD_SPEC_T g_lcd_GC9307[];
.....
```

- Configure LCD related information in LCM_CFG_INFO_T s_main_lcm_cfg_tab[].
Path: **MS_Code\MS_Customize\source\product\config\fp6531_bar\lcm_cfg_info.c**

```
LOCAL LCM_CFG_INFO_T s_main_lcm_cfg_tab[] = {
    #ifdef MAIN_LCM_INTERFACE_SPI
    #ifdef FPGA_VERIFICATION
    { LCD_DRV_ID_ST7789H2, 0, 0, 0, SPI1_O_CS0, SPI1_O_CD, LCD_CTRL_MODULE_SPI, (LCD_SPEC_T_PTR) &g_lcd_ST7789H2, LCD_BACKLIGHT_TYPE_BB_ANALOG, 0, 0, 0, NULL},
    #else
    { LCD_DRV_ID_GC9305, 0, 0, 0, SPI1_O_CS0, SPI1_O_CD, LCD_CTRL_MODULE_SPI, (LCD_SPEC_T_PTR) &g_lcd_GC9305, LCD_BACKLIGHT_TYPE_BB_ANALOG, 0, 0, 0, NULL},
    #endif
    #else
    #ifdef FPGA_VERIFICATION
    { LCD_DRV_ID_ST7789H2, 0, 0, 0, 0, 0, LCD_CTRL_MODULE_LCM, (LCD_SPEC_T_PTR) &g_lcd_ST7789H2, LCD_BACKLIGHT_TYPE_BB_ANALOG, 0, 0, 0, NULL},
    #else
    { LCD_DRV_ID_GC9108, 0, 0, 0, 0, 0, LCD_CTRL_MODULE_LCM, (LCD_SPEC_T_PTR) &g_lcd_GC9108, LCD_BACKLIGHT_TYPE_BB_ANALOG, 0, 0, 0, NULL},
    #endif
    #endif
};
/*lint --restore*/

#ifdef PRODUCT_DRIVER_SUB_LCD_ENABLE
LOCAL LCM_CFG_INFO_T s_sub_lcm_cfg_tab[] = {
    { LCD_DRV_ID_ILI9328, 0, 0, 0, 0, 0, LCD_CTRL_MODULE_LCD, (LCD_SPEC_T_PTR) &g_lcd_ILI9328, LCD_BACKLIGHT_TYPE_BB_ANALOG, 0, 0, 0, NULL},
};
#endif
```

Configure the following parameters in LCM_CFG_INFO_T s_main_lcm_cfg_tab[]:

Screen IC

uint32	dev_id;	// ID of LCD driver product IC
--------	---------	--------------------------------

ADC

uint32	adc_chn;	// ADC channel connected with LCM
uint16	adc_min;	// Min. value of the valid range
uint16	adc_max;	// Max. value of the valid range GC9307 Device Address

spec_info

uint16	cs_pin;	// cs pin connected with LCM
uint16	cd_pin;	// cd pin connected with LCM
LCD_CTRL_MODULE_E	controller;	
LCD_SPEC_T_PTR	spec_ptr;	

bl_control backlight information

LCD_BACKLIGHT_TYPE_E	type;
uint32	param;
uint32	bl_min;

uint32

bl_max;

- Modify the MAINLCD_DEV_SIZE value in the main mk file according to the used LCD screen size.

```
MAINLCD_DEV_SIZE = 128X160
```

```
MAINLCD_SIZE = 128X160
```

```
MAINLCM_INTERFACE = SPI
```

```
LCD_SPI = 4WIRE_8BIT_1DATA
```

```
SPI_DATA2_SEL = DI
```

```
SUBLCM_INTERFACE = NONE
```

```
LCD_DATA_WIDTH = 8BIT
```

```
# Option1:TRUE
# Option2:FALSE
### Resolution Of Main LCD Device, MAINLCD_SIZE is the size of MMI
# Option1:128X160 #define MAINLCD_DEV_SIZE_128X160
# Option2:176X220 #define MAINLCD_DEV_SIZE_176X220
# Option3:240X320 #define MAINLCD_DEV_SIZE_240X320
# Option4:320X480 #define MAINLCD_DEV_SIZE_320X480

### Resolution Of Main LCD:
# Option1:128X160 #define MAINLCD_SIZE_128X160
# Option2:176X220 #define MAINLCD_SIZE_176X220
# Option3:240X320 #define MAINLCD_SIZE_240X320
# Option4:320X480 #define MAINLCD_SIZE_320X480

#### LCM interface support
# Option1:LCM
# Option2:SPI
# Option3:NONE

#### LCD SPI mode support
# Option1:3WIRE_9BIT_1DATA
# Option2:3WIRE_9BIT_2DATA
# Option3:4WIRE_8BIT_1DATA
# Option4:4WIRE_8BIT_2DATA

#### LCD SPI data2 line select
# Option1:CI
# Option2:DI

#### Sub LCM interface support
# Option1:LCM
# Option2:SPI
# Option3:NONE

### FPGA LCM LCD 8BIT
```

2.2 Debugging method

LCD debugging requires you to capture serial interface log or use TRACE32 for single-step debugging.

2.2.1 LCD ID not identified

After adding the LCD driver, if the system identifies LCD ID, the identified ID value will be printed in the log of the interface for identifying ID, as shown below:

```
SCI_TraceLow("lcd: GC9307_ReadID: id1- >0x%0x, id2- >0x%0x, id3- >0x%0x\n", id[3], id[2], id[1]);
```

If LCD ID is not identified, debug the LCD driver in the following ways:

- Increase DEFAULT_RESET_TIME due to insufficient default Reset time.
- Confirm whether the SPI bus setting mode or SPI rate is correct according to the hardware configuration requirements.

2.2.2 Abnormal LCD display

If LCD ID is normal, but the LCD display is abnormal, such as white screen, splash screen, blurred screen, abnormal color, or incomplete display, debug the LCD driver in the following ways:

- White screen: check whether the LCD time sequence and the driver file are correct and whether the LCD Reset time is enough.
- Splash screen:
 - The flashing white screen during wake up or startup is caused by the early lighting of the backlight.
 - The screen flashes during normal use, which may be caused by backlight.

- If Electro Static Discharge (ESD) protection is enabled, the initialization code downloaded when ESD checknormal is executed may be incorrect.
- Blurred screen: LCD initialization is successful, but the screen has not refreshed the data. You need to check the driver parameter configuration.
- Split screen: check the configuration of the screen driver register (such as 0x44 of tft_GC9307) and open TE to prevent split screen.
- Abnormal color: check the configuration of the screen driver register (0x36 of tft_GC9307) and confirm whether the initialization parameters of the screen are correct with the screen manufacturer.
- Incomplete display: check the configuration of the screen driver register (0x2a and 0x2b of tft_GC9307).
- Display direction: check the configuration of screen driver register (0x36). 0x36 defines the read/write direction of the frame scanning.